Project Based Learning Report

On

AuthentiCheck: A Mobile-First Real-Time E-Commerce Fraud Auditor

Submitted By

54- Rudrek Rokade 57- Pragati Sanap

70- Atharva Thakare 74- Yash Yadav

Class: TY AI&DS - B

Under the Guidance of

Prof. U. M Rane

Department of Artificial Intelligence and Data Science

K. K. Wagh Institute of Engineering Education & Research

Hirabai Haridas Vidyanagari, Amrutdham, Panchavati,

Nashik - 422 003

Date of Submission: 24 September 2026

Abstract

AuthentiCheck is a serverless, mobile-first utility application designed to provide a real-time, explainable authenticity and fraud assessment for e-commerce product listings. The system focuses on signals associated with counterfeit listings, manipulated review ecosystems, deceptive merchant storefronts, and misleading price patterns. Its central design choice is to analyze the live product page at the moment of evaluation instead of depending on a stale central fraud database. A shopper shares an Amazon or Flipkart product URL directly from the native mobile OS share sheet, and AuthentiCheck receives the URL through Flutter sharing-intent integration. Users can also paste a product URL directly into the application home screen for immediate evaluation. The app then requests live HTML through ScraperAPI, performs client-side DOM pruning with Dart HTML parsing tools, and keeps only semantic information such as product title, price, seller credentials, ratings, and selected review text. The lightweight payload is sent to a high-speed LLM endpoint such as Groq Cloud, where prompt-constrained heuristic inference produces an authenticity score, a categorical verdict, a price anomaly indicator, and a list of evidence-backed risk factors. The output is then parsed as deterministic JSON and rendered in a high-contrast brutalist interface that exposes the reasoning rather than hiding it behind a single label. The design targets a 3–4 second end-to-end inference experience and uses aggressive sanitization to target more than 95% reduction in payload size before LLM inference. The project also emphasizes graceful failure, transparent limitations, and a clean separation between evidence extraction and AI reasoning. Because the supplied project context does not include a labelled benchmark dataset or measured accuracy logs, the evaluation in this report focuses on functional architecture, acceptance criteria, live prototype walkthroughs, and a reproducible prototype testing plan rather than invented performance claims.

Keywords:
E-commerce fraud detection, counterfeit listings, fake review detection, mobile-first security, Flutter, ScraperAPI, DOM sanitization, large language models, structured JSON, explainable AI.

Table of Contents

1. Introduction	4
2. Problem Statement	6
3. Research and Background	7
4. Project Design and Planning	8
5. Implementation	11
6. Results and Evaluation	14
7. Learning Outcomes	18
8. Future Scope	19
9. Conclusion	20
10. References	21
11. Appendices	22

1. Introduction

1.1 Overview of the Project

AuthentiCheck is conceived as a lightweight mobile utility that sits beside the shopping experience instead of replacing the marketplace application. The user can browse normally on an e-commerce platform, open the native Share menu, select AuthentiCheck, and return an assessment based on the exact product-page context available at that moment. The project therefore treats authenticity checking as an on-demand audit rather than a permanent background service.

The application is intentionally mobile-first. The interaction is optimized for a shopper who is already looking at a product on a phone and needs a low-friction decision aid before purchasing. Instead of asking the user to copy a link, switch to another website, and manually enter information, the system receives the URL through the operating system share mechanism. This reduces interaction cost and keeps the audit close to the moment of purchase. In addition, the Flutter-based mobile application also supports pasting a product URL directly into the home screen input box so users can audit links from any source at any time.

1.2 Motivation and Project Choice

The project addresses a practical trust problem in online shopping: a listing can appear convincing while different parts of the page contain weak or suspicious evidence. Rating counts, repeated review language, seller age, seller identity, and price presentation can each be misleading in isolation. AuthentiCheck is designed to combine these heterogeneous cues into a single explainable assessment that still exposes the underlying evidence ledger.

1.3 Scope

- Primary platforms: product pages from marketplaces such as Amazon and Flipkart, delivered through shared URLs or direct URL paste input.
- Primary evidence: product title, displayed price, seller information, ratings, and selected review text visible in the fetched page.
- Primary output: authenticity score, categorical verdict, price anomaly flag, and human-readable risk factors in a mobile dashboard.
- Architecture scope: client-side orchestration with third-party scraping and LLM APIs; no custom application backend.
- Out of scope for the supplied project context: payment processing, account takeover protection, legal adjudication of counterfeit goods, and guaranteed ground-truth authenticity.

1.4 Project Purpose and Main Features

The core idea behind the project is simple: shoppers often rely on public product pages that may contain misleading pricing, suspicious seller patterns, inflated reviews, or copied product descriptions. AuthentiCheck gives users a quick confidence check before they purchase an item. The app focuses on:

- Analyzing product URLs from marketplaces such as Amazon and Flipkart.
- Extracting visible product information from the page and stripping non-visible HTML markup.
- Checking for indicators like price anomalies and suspicious listing or review content.
- Returning an easy-to-read score and verdict alongside detailed risk signals.
- Supporting share-based URL intake from outside the app as well as direct URL input.

The main features implemented in the AuthentiCheck application include:

- Product URL input (PRODUCT_URL) and quick single-tap analysis (ANALYZE).
- Share integration from other apps or browsers.
- Real-time loading states and terminal-style dashboard design (> Fetching data from source..., > Fetching data from ScraperAPI..., > Raw data acquired., > Executing LLM heuristic analysis...).
- Web scraping via ScraperAPI.
- AI-powered assessment via Groq-based LLM.

- Structured output with authenticity score (0–100), verdict summary, price anomaly flag (PRICE_ANOMALY), and list of risk factors (RISK_FACTORS).

2. Problem Statement

2.1 Definition of the Problem

Online shoppers are asked to make purchase decisions using information that may itself have been manipulated. Counterfeit sellers can benefit from product pages with convincing titles and images, inherited positive review history, artificial ratings, or discount presentations that do not reflect a stable market price. Review ecosystems can also contain repeated or coordinated language that looks persuasive when viewed individually but becomes suspicious when considered as a group.

Existing fraud-detection approaches often create friction for mobile shoppers when they depend on browser extensions, pre-computed databases, or workflows outside the marketplace app. AuthentiCheck addresses this gap by evaluating the live listing context and returning a compact, evidence-oriented assessment immediately before the purchase decision.

2.2 Objectives

1. Capture product URLs from the native share sheet without copy-paste interaction, while also supporting direct URL paste input on the home screen.
2. Fetch live listing HTML through a scraping proxy capable of handling marketplace anti-bot constraints.
3. Prune irrelevant DOM content on the client and retain only high-value semantic evidence.
4. Use LLM-based heuristic inference to identify suspicious review, seller, listing, and pricing signals.
5. Enforce a deterministic JSON response containing an authenticity score, verdict, price anomaly status, and risk factors.
6. Present results in a fast, high-contrast mobile interface that explains why a listing was flagged.
7. Design the system so that there is no custom backend hosting layer in the project architecture.

2.3 Existing Approach and Limitations

Aspect	Common Limitation	AuthentiCheck Response
Workflow	Manual URL copy/paste or separate browser workflow	Native share-sheet interception + direct paste input
Data freshness	Static/pre-computed fraud records can become stale	Live page fetch at audit time
Page size	Raw HTML is too large for direct LLM processing	Aggressive DOM pruning on device

Aspect	Common Limitation	AuthentiCheck Response
Infrastructure	Centralized fraud database may require ongoing hosting	Client-side orchestration plus external APIs
Explainability	Single fraud label gives little context	Evidence Ledger with risk factors and price anomaly flag

3. Research and Background

3.1 Theoretical Foundation

The project is based on the idea that authenticity is better treated as an evidence-integration problem than as a single observable field. Product pages expose multiple signals - linguistic patterns, reviewer behavior, seller context, price presentation, and platform metadata. The application therefore combines rule-like heuristics with language-model reasoning rather than depending on one feature alone.

Research on fake-review detection commonly considers review characteristics, textual characteristics, reviewer characteristics, seller characteristics, and platform characteristics as distinct cue families. A systematic review of consumer-side fake-review research grouped cues into these five categories, highlighting that users evaluate credibility using more than the wording of a single review. [5]

3.2 Fake Reviews and Deceptive Signals

Prior work shows that automated fake-review detection is difficult and that content, behavioral, and interrelationship features can all contribute to detection. A 2022 state-of-the-art review also identified data acquisition and feature-design challenges, especially when large labelled datasets are unavailable. [6] More recent work has examined LLM-based review analysis, supporting the project choice to use an LLM as a reasoning component while also emphasizing generalization and evaluation limits. [4]

3.3 LLM Reasoning and Structured Output

AuthentiCheck does not ask the language model to invent a free-form essay. It asks for a strict schema with core outputs: authenticity_score, verdict, price_anomaly, and risk_factors. Structured output interfaces are designed for this type of downstream integration because they constrain model responses to a JSON schema rather than requiring brittle natural-language parsing. Current Groq documentation describes JSON mode and JSON Schema structured outputs, including a strict mode for supported models. [3]

3.4 Mobile HTML Parsing

The Dart html package provides HTML5 parsing and a DOM-like tree API. This makes it suitable for a client side sanitization stage that can remove script, style, navigation, and other non-semantic content before sending data to the reasoning service. [2] The project uses this parser as a data-reduction step, not as a full browser engine.

3.5 Share-Intent Integration

The receive_sharing_intent Flutter package is designed to receive shared text and URLs from other applications and supports Android and iOS. This directly matches the interaction requirement of selecting AuthentiCheck from the native mobile Share menu. [1]

4. Project Design and Planning

4.1 System Architecture

The architecture is intentionally thin. The mobile application performs orchestration, URL intake, HTML sanitization, response parsing, and presentation. External services provide two specialized capabilities: live page retrieval and high-speed LLM inference. No custom REST API, database, authentication server, or scheduled backend process is required by the described project architecture.

Stage	Component	Primary Responsibility	Output
1	Native Share Sheet / Home Input	Receive product URL from marketplace app or user paste	URL
2	Flutter Intent Layer (HomeScreen)	Normalize shared payload and validate host	Validated URL
3	ScraperAPI (ScraperService)	Fetch live marketplace HTML	Raw HTML
4	Dart HTML Parser	Prune irrelevant DOM and extract semantic text	Sanitized evidence text
5	LLM Endpoint (LLMService)	Infer risk from evidence using strict prompt + schema	JSON assessment
6	Deterministic Parser (AuthenticityResult)	Validate schema and handle failures	Typed assessment object
7	Mobile UI (BrutalistTheme)	Render score, verdict, price anomaly, terminal log, and evidence ledger	User-facing report

4.2 Project Timeline

Phase	Milestone	Planned Activities
Phase 1	Problem framing	Threat model, requirements, scope, user flow
Phase 2	Mobile intake	Flutter app shell, share intent, URL validation
Phase 3	Data acquisition	ScraperAPI request path, error handling
Phase 4	Sanitization	DOM pruning, semantic extraction, payload reduction
Phase 5	LLM inference	Prompt design, JSON schema, heuristic categories
Phase 6	UI integration	Terminal progress, dashboard, Evidence Ledger
Phase 7	Evaluation	Functional cases, failure cases, latency and payload measurements

4.3 Task Allocation

Work Package	Core Responsibility	Deliverable
Research & requirements	Define fraud cues and user workflow	Requirement specification
Flutter/mobile	Share sheet, navigation, state management	Mobile interaction layer
Data pipeline	Scraping, parsing, sanitization	Evidence extraction module
AI integration	Prompt, schema, response parsing	LLM inference module
UI/UX	Brutalist visual language, logs, ledger	Assessment dashboard
Testing & documentation	Failure cases, acceptance criteria, report	Validation record + documentation

4.4 Tools and Technologies

Technology	Role in Project
Flutter/Dart	Cross-platform mobile application development and orchestration
Material Design UI	Core widget foundation styled with custom brutalist industrial theme
receive_sharing_intent	Native share URL intake
http package	HTTP requests for scraping and AI chat completion access
flutter_dotenv	Loading runtime environment variables (SCRAPER_API_KEY, GROQ_API_KEY)
Dart html package	HTML5 parsing and DOM-style sanitization
ScraperAPI	Live HTML acquisition through scraping infrastructure
Groq Cloud / OpenAI-compatible chat completion API	Heuristic fraud inference and structured response generation
JSON Schema	Deterministic contract between LLM output and mobile UI
Brutalist UI approach	High-contrast, information-dense result presentation

4.5 Project Structure

The application is organized into the following source files and directories:

- lib/main.dart: App entry point and app initialization.
- lib/screens/home_screen.dart: Main UI screen that handles input, loading states, result display, and share intent handling.

- lib/services/scraper_service.dart: Fetches product page data from ScraperAPI and strips non-visible HTML content before sending it to the LLM.
- lib/services/llm_service.dart: Sends the cleaned product content to the Groq API and parses the JSON response.
- lib/models/authenticity_result.dart: Model class representing the authenticity analysis response.
- lib/theme/brutalist_theme.dart: Custom visual styling for the app's harsh, industrial dashboard look.
- android/, ios/, linux/, macos/, web/, windows/: Platform-specific Flutter project files created for multi-platform support.

4.6 Key Data Flow and How the App Works

The user flow is straightforward:

1. The user pastes a product URL or shares a link from another app.
2. The app captures the URL in the home screen.
3. The scraper fetches the product page content.
4. HTML is cleaned and reduced to visible text.
5. The cleaned data is sent to an LLM prompt.
6. The model returns structured JSON containing the score, verdict, price anomaly status, and risk factors.
7. The UI renders the final results in a bold, terminal-inspired layout.

```
Product URL
   ↓
HomeScreen
   ↓
ScraperService
   ↓
HTML sanitization
   ↓
LLM analysis
   ↓
AuthenticityResult JSON
   ↓
Dashboard UI
```

5. Implementation

5.1 Native URL Interception

The user begins inside the marketplace application or directly on the AuthentiCheck home screen. When a product page is open, the user invokes the native share menu and selects AuthentiCheck. The share-intent layer receives the shared URL and forwards it into the application state machine. Alternatively, the user can paste a product URL into the PRODUCT_URL input box on HomeScreen and press ANALYZE. The URL is then normalized and checked against the supported marketplace hosts before any network request is attempted.

```
Shared URL
-> receive_sharing_intent / HomeScreen input
-> validate scheme + host
-> normalize URL
-> start audit state
```

5.2 Live Proxy Scraping

A direct client-side HTTP request is made by ScraperService to ScraperAPI with the marketplace URL as the target. The proxy service is responsible for the scraping layer, including proxy handling and other anti-bot techniques supported by its API. The application itself remains small because it does not maintain a scraper fleet or proxy pool. [7]

The implementation should treat a scrape failure as a first-class state. Timeouts, blocked pages, empty responses, unsupported marketplaces, or unexpected HTML should produce an actionable error rather than an empty risk score. This is important because an LLM cannot produce trustworthy reasoning when the underlying page evidence is missing.

5.3 On-Device DOM Sanitization

Raw product pages can contain large amounts of scripts, style definitions, navigation menus, tracking markup, and duplicated interface content. The sanitizer parses the HTML into a DOM-like structure and removes low value nodes before extracting text. The target semantic set contains product title, displayed price, seller credentials, rating summaries, and selected review content. The design target is a payload reduction of more than 95%, reducing tokens, network transfer, and downstream processing cost.

Raw Page Element	Action	Reason
script/style	Remove	Non-semantic implementation code
navigation/footer	Remove	Repeated UI content and low-value text
product title	Keep	Core product identity
price / discount	Keep	Pricing anomaly signal

Raw Page Element	Action	Reason
seller details	Keep	Merchant credibility signal
rating summary	Keep	Aggregate reputation signal
review text/metadata	Keep selected content	Textual and temporal anomaly signals

5.4 LLM Heuristic Inference

The sanitized content is placed into a strict prompt that defines the LLM as an e-commerce fraud investigator. The prompt directs the model to reason from observable evidence, identify suspicious patterns, and avoid making claims that are not supported by the supplied page context. Heuristic families include repeated review phrasing, unusual review clusters, seller-newness indicators when available, aggressive discount patterns, inconsistent seller/product information, and other signals explicitly present in the extracted text.

5.5 Deterministic JSON Parsing

The LLM response is required to conform to a fixed schema represented by AuthenticityResult. The application then validates field presence, data types, and expected ranges before rendering. A malformed or incomplete answer is treated as a parsing failure rather than silently displayed as valid evidence.

```
{
   "authenticity_score": 0-100,
   "verdict": "SAFE | CAUTION | HIGH_RISK",
   "price_anomaly": "NORMAL | SUSPICIOUS | HIGH_DISCOUNT",
   "risk_factors": ["evidence-backed factor 1", "evidence-backed factor 2"] }
```

5.6 User Interface and Interaction Design

The user interface deliberately avoids generic enterprise dashboard styling. The project uses a high-contrast brutalist visual language (lib/theme/brutalist_theme.dart), large numeric status, compact labels, and a terminal-style audit log during the 3–4 second inference window. Once a response is ready, the primary score is paired with a verdict summary banner, a PRICE_ANOMALY status block, and a RISK_FACTORS Evidence Ledger so that users can see the specific reasons that influenced the assessment, followed by a SCAN_ANOTHER_URL action button. This design aligns the interface with the project goal of explainability: the system shows evidence rather than only asserting a percentage.

5.7 Security and Failure Handling

- Do not embed long-lived high-privilege service credentials in the UI or source package; API access should use the least-privileged mechanism available to the deployment.
- Reject unsupported URL schemes and hosts before scraping.
- Timeout third-party requests and surface service failure explicitly.
- Treat missing evidence as uncertainty, not as proof of authenticity or fraud.
- Validate all structured LLM fields before using them in the UI.

5.8 Environment Setup and Setup Instructions

This app expects a .env file in the project root with API keys configured. These keys are loaded at app startup via flutter_dotenv:

```
SCRAPER_API_KEY=your_scraper_api_key
GROQ_API_KEY=your_groq_api_key
```

Setup instructions for running the application:

1. Install Flutter dependencies:

```
flutter pub get
```

2. Create a .env file in the project root and add the required keys.
3. Run the app:

```
flutter run
```

6. Results and Evaluation

6.1 Final Product / Outcome

The project outcome is a mobile utility architecture in which a shopper can submit a marketplace product URL through the system share sheet or direct home-screen input and receive a compact risk assessment derived from the live listing. The deliverable integrates the complete conceptual path from URL intake to user-facing explanation: share intent, live retrieval, semantic extraction, LLM reasoning, structured parsing, and visual presentation.

The most important product characteristic is the Evidence Ledger. Instead of reducing the output to a statement such as "70% fake," the interface is designed to surface observations such as repeated review wording, unusual review timing, seller-age indicators, or pricing anomalies when those signals are actually visible in the source page. This makes the result easier to inspect and more appropriate for a decision-support tool.

6.1.1 Prototype Execution Walkthrough (Application Screenshots)

Figures 1 through 6 illustrate the working AuthentiCheck mobile application across the complete audit lifecycle, from URL intake and live terminal status updates to the rendered authenticity score, price anomaly status, and risk factors ledger.

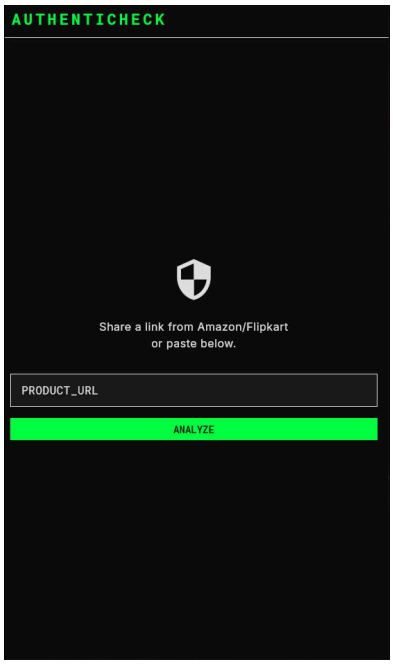


Figure 1: Home Screen (Idle). Displays the brutalist dark interface, shield icon, prompt to share or paste an Amazon/Flipkart link, PRODUCT_URL field, and ANALYZE button.

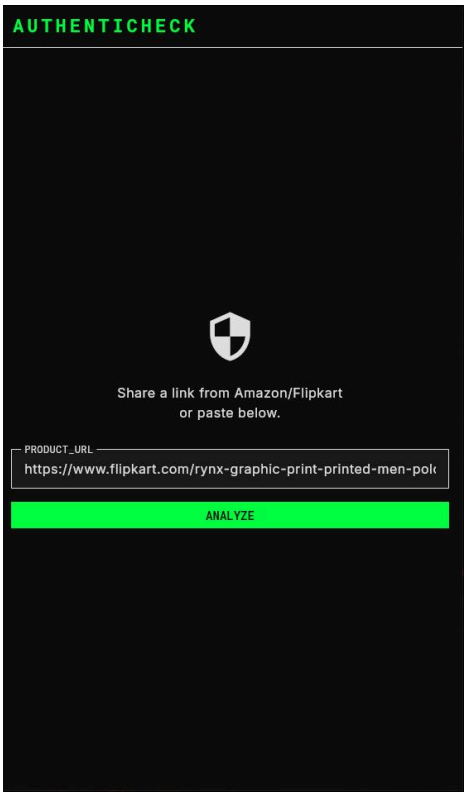


Figure 2: Product URL Intake. A live Flipkart product URL (https://www.flipkart.com/rynx graphic-print-printed-men polo...) is populated in PRODUCT_URL for analysis.

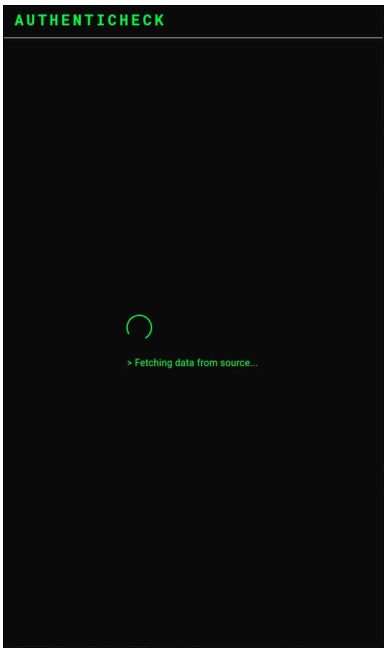


Figure 3: Audit Initiation Log. Terminal progress screen displaying the active spinner and log line: > Fetching data from source....

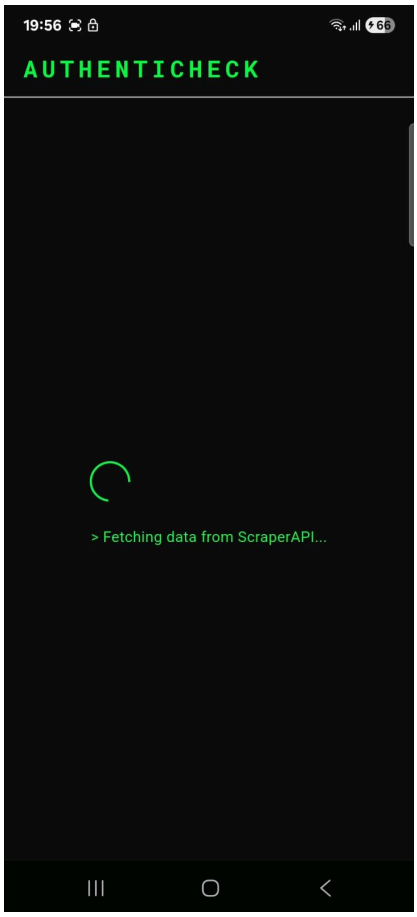


Figure 4: Live Proxy Scraping. ScraperService retrieves the live product HTML via ScraperAPI, logging > Fetching data from ScraperAPI....

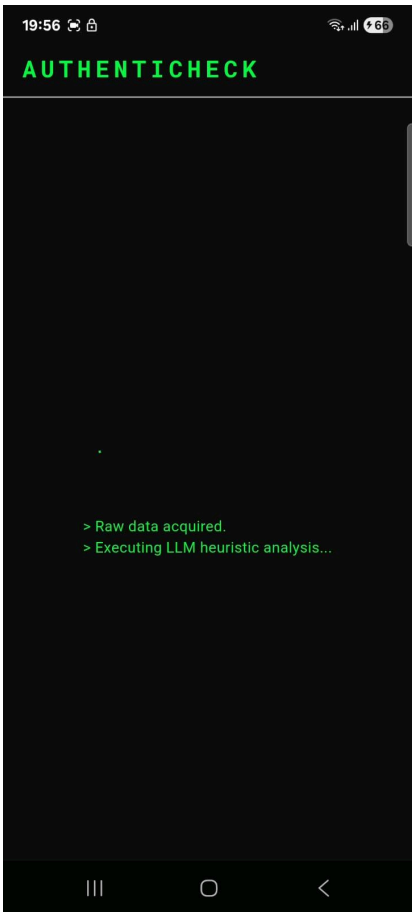


Figure 5: DOM Pruning & LLM Analysis. After HTML sanitization, the terminal logs > Raw data acquired. and > Executing LLM heuristic analysis....

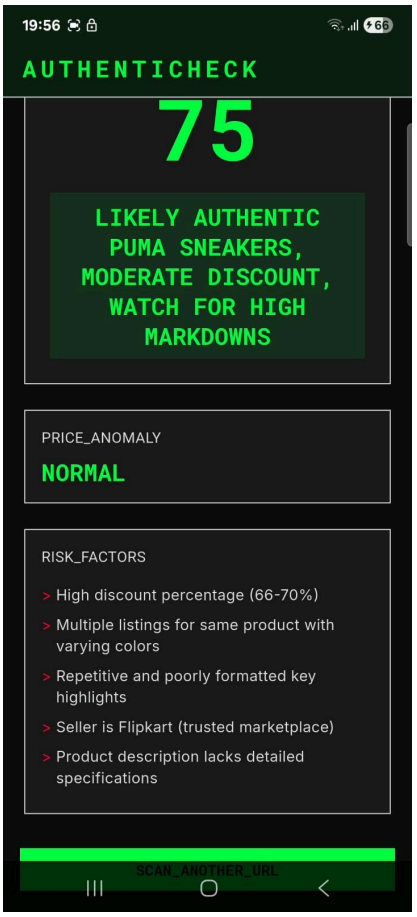


Figure 6: Rendered Assessment. Shows Score 75, verdict summary, PRICE_ANOMALY: NORMAL, five RISK_FACTORS, and SCAN_ANOTHER_URL.

6.2 Functional Evaluation Plan

Test Case	Expected Behaviour	Acceptance Condition
TC-01 Valid shared URL	App opens audit flow from marketplace share menu or direct paste	URL captured without manual retyping
TC-02 Invalid URL	App rejects unsupported or malformed URL	Clear validation message; no LLM call
TC-03 Scrape success	HTML returned and sanitization starts	Semantic evidence extracted
TC-04 Scrape failure	Timeout/block/empty response handled	User sees explicit failure state
TC-05 Sanitization	Scripts, styles, navigation removed	Output contains target evidence fields

Test Case	Expected Behaviour	Acceptance Condition
TC-06 LLM valid JSON	Response conforms to schema	Score, verdict, price anomaly, risk factors rendered
TC-07 LLM malformed JSON	Parser rejects invalid structure	Safe fallback state shown
TC-08 Evidence missing	Insufficient information detected	Result communicates uncertainty
TC-09 UI logging	Progress states update during inference	No frozen screen; completion is visible
TC-10 End-to-end	Valid page flows through all stages	Assessment reaches dashboard successfully

6.3 Performance Evaluation

The supplied project context specifies a 3–4 second inference experience and a payload-reduction target above 95%. These values are treated here as design targets, not independently measured benchmark results. A complete evaluation should record end-to-end latency, scrape latency, sanitization time, token count before and after pruning, failure rate, and structured-output validity over a defined test set.

Metric	Definition	Target/Evaluation Rule
End-to-end latency	Share action to rendered assessment	Target: 3-4 seconds under representative network conditions
Payload reduction	1 - sanitized size / raw size	Target: >95% by project design
JSON validity	Valid schema responses / total responses	Should be measured across repeated runs
Evidence completeness	Required evidence fields present when source exposes them	Report by test-case category
Failure handling	Share of known failure cases with explicit user feedback	No silent success on missing evidence

6.4 Interpretation and Limitations

An LLM-generated authenticity score is a heuristic assessment of the supplied evidence; it is not a laboratory verification that a physical product is genuine. The result can be affected by incomplete scraping, marketplace layout changes, ambiguous seller information, review deletion, language variation, or adversarial manipulation. The system is therefore best interpreted as a pre-purchase warning and explanation layer rather than an absolute truth oracle.

The project context also does not provide a labelled ground-truth dataset, confusion matrix, or independently measured accuracy. Consequently, no fabricated precision, recall, F1 score, or detection rate is reported here.

Additional prototype notes and operational limitations:

- The app depends on external APIs, so internet access and valid credentials are required.
- The scraper and AI services are best-effort tools and may fail if a page is blocked or rate-limited.
- Results are heuristic and should be treated as guidance rather than guaranteed proof of authenticity.
- The project is a prototype and can be extended with more advanced checks such as seller reputation, review authenticity detection, and image analysis.

6.5 Risk and Mitigation Considerations

Risk	Potential Effect	Mitigation
Marketplace HTML changes	Selectors may stop finding evidence	Keep extraction rules modular; maintain regression fixtures
Scraper or LLM outage	Audit cannot obtain fresh evidence	Show explicit failure state and allow retry
Adversarial review text	Manipulated language may influence inference	Use multiple evidence categories and evidence backed factors
Prompt injection in page text	Page content may attempt to alter model instructions	Separate instructions from untrusted content and validate schema
Weak or missing evidence	Score may be misleading	Surface uncertainty and avoid definitive claims
Third-party limits	Requests may exceed service allowances	Sanitize before inference, rate-limit, and monitor usage

7. Learning Outcomes

7.1 Skills Acquired

- Mobile integration: designing share-intent workflows and state transitions in Flutter.
- Data engineering: reducing raw web documents to compact semantic evidence suitable for downstream reasoning.
- API integration: coordinating third-party scraping and LLM services with runtime environment configuration (flutter_dotenv), timeout, and failure states.
- Prompt engineering: defining task-specific heuristics and structured response contracts.
- Software design: separating acquisition (ScraperService), transformation, inference (LLMService), parsing (AuthenticityResult), and presentation (HomeScreen, BrutalistTheme) into distinct responsibilities.
- UX design: communicating uncertainty and evidence through a compact mobile interface.

7.2 Knowledge Gained

The project connects concepts from mobile computing, web parsing, applied AI, and cybersecurity. In particular, it demonstrates why real-time data handling matters when the source page can change frequently, why data reduction is important before LLM inference, and why a strict machine-readable contract is preferable to loosely formatted model output in a production-like application.

7.3 Critical Reflection

A key learning from the architecture is that the LLM is only one component in a longer evidence pipeline. The quality of the final result depends on the freshness and completeness of the extracted page. This shifts the engineering focus away from model selection alone and toward reliable orchestration, input validation, data sanitization, structured outputs, and clear failure states. Another learning is that explainability can be treated as a user-interface requirement rather than an afterthought: the Evidence Ledger is part of the product design, not just a debug console.

8. Future Scope

8.1 Potential Improvements

- Multi-marketplace adapters for additional e-commerce sites and regional storefront variants.
- Historical price tracking and comparing prices across multiple marketplaces so that discount claims can be compared with a longer price baseline.
- Seller-age, seller history checks, and seller-identity enrichment when the marketplace exposes reliable metadata.
- Detecting suspicious review patterns and multilingual and cross-lingual review analysis for regional marketplaces.
- Image-level signals such as logo mismatch, packaging inconsistency, or duplicate listing images.
- A calibrated confidence/uncertainty layer that distinguishes strong evidence from weak evidence.
- Adding a history of prior scan results and storing analysis in a local database alongside local caching of non-sensitive intermediate artifacts to improve responsiveness while respecting platform terms and privacy requirements.
- Improving UI with richer charts and explanation layers.
- Automated regression testing against captured, permissioned test fixtures to detect changes in marketplace HTML structure.

8.2 Research Extensions

Future experimentation could compare the LLM heuristic approach with classical supervised models, transformer classifiers, graph-based reviewer-network features, and hybrid systems. Recent literature continues to explore explainability, cross-domain generalization, adversarial adaptation, and robust handling of AI generated reviews, making these suitable themes for a later research phase. [4][5][6]

8.3 Deployment Considerations

A production deployment would need explicit review of marketplace terms, scraping permissions, privacy practices, API cost controls, secret management, rate limiting, and model governance. The current architecture minimizes custom hosting but remains dependent on external providers for live page acquisition and LLM inference, so service limits and availability should be treated as architectural dependencies rather than hidden assumptions.

9. Conclusion

9.1 Summary of Findings

AuthentiCheck presents a practical architecture for mobile e-commerce trust assistance. Its central idea is to evaluate the live product page at the time of purchase, reduce the page to essential evidence on the device, and use a high-speed LLM to reason over that evidence under a strict JSON contract. The approach removes unnecessary interaction steps, avoids a custom backend hosting layer, and emphasizes explainable output through an Evidence Ledger.

The project also demonstrates an important systems principle: AI reasoning is only as useful as the evidence pipeline that feeds it. Native URL interception, robust scraping, DOM sanitization, schema validation, and failure handling are therefore first-class parts of the solution. AuthentiCheck is a practical Flutter app that blends scraping and AI analysis to provide a quick counterfeit-risk assessment for online products. It is a strong foundation for an e-commerce trust and safety tool, especially when expanded with more seller, pricing, and review-based validation logic. The final design is intended as a consumer decision-support tool, not as definitive proof of whether a physical product is genuine.

9.2 Key Takeaways

- Live context can reduce reliance on stale, pre-computed fraud information.
- Client-side sanitization can keep LLM inputs compact and focused on semantic evidence
- Structured JSON makes AI output easier to validate and integrate into application logic.
- Explainability improves when the interface shows specific evidence instead of only a single risk label.
- Performance, accuracy, and reliability should be measured with a defined test set before making quantitative claims.

10. References

[1] receive_sharing_intent. (2026). Flutter package documentation. pub.dev. https://pub.dev/packages/receive_sharin g_intent

[2] html. (2026). Dart package documentation: HTML5 parsing and DOM APIs. pub.dev. https://pub.dev/packages/h tml

[3] Groq. (2026). Structured Outputs and API Reference. Groq Documentation. https://console.groq.com/docs/struct ured-outputs; https://console.groq.com/docs/api-reference

[4] Bhangale, S., & Roy, P. K. (2025). Is it genuine or fake? Analyzing e-commerce reviews using large language models. Knowledge-Based Systems, 330, 114556. https://doi.org/10.1016/j.knosys.2025.114556

[5] Harrison-Walker, L. J., & Jiang, Y. (2023). Suspicion of online product reviews as fake: Cues and consequences. Journal of Business Research, 113780. https://doi.org/10.1016/j.jbusres.2023.113780

[6] Research on false review detection methods: A state-of-the-art review. (2022). Journal of King Saud University - Computer and Information Sciences, 34(9), 7530-7546. https://doi.org/10.1016/j.jksuci.2021.07.021

[7] ScraperAPI. (2026). Web scraping solutions and API documentation. https://www.scraperapi.com/solutions/

Note: The project report integrates the supplied AuthentiCheck project description with the referenced technical documentation and background literature. Accessed September 2026.

11. Appendices

Appendix A - End-to-End Pipeline Summary

A1. Open product in Amazon/Flipkart or another supported marketplace.
A2. Use the native mobile Share action and select AuthentiCheck (or paste the product URL directly in the app).
A3. Receive and validate the URL using share-intent integration or home screen input.
A4. Request the live page through ScraperAPI.
A5. Parse and prune the DOM on the device; retain semantic evidence.
A6. Send sanitized text to a high-speed LLM with a strict schema prompt.
A7. Validate JSON fields and render the score, verdict, price anomaly status, and risk factors.
A8. Show the Evidence Ledger and allow the shopper to make an informed purchase decision.

Appendix B - Representative Pseudocode

```
onSharedUrl(url):
if !isSupportedMarketplace(url):
showError('Unsupported product URL')
return

rawHtml = await scrapeLivePage(url)
evidence = sanitizeDom(rawHtml)

result = await inferRisk(
evidence,
schema = [score, verdict, price_anomaly, risk_factors]
)

assessment = parseAndValidate(result)
renderAssessment(assessment)
```

Appendix C - Example Structured Assessment

```
{
"authenticity_score": 42,
"verdict": "HIGH_RISK",
"risk_factors": [
"Seller evidence appears limited or recently established",
"Multiple reviews share unusually similar phrasing",
"Displayed discount is unusually steep relative to available context" ]
}
```

Note: This is an illustrative schema example, not a measured assessment of a real listing.

Live Prototype Assessment Output (Captured in Figure 6):

```
{
"authenticity_score": 75,
"verdict": "LIKELY AUTHENTIC PUMA SNEAKERS, MODERATE DISCOUNT, WATCH FOR HIGH
   MARKDOWNS",
"price_anomaly": "NORMAL",
"risk_factors": [
"High discount percentage (66-70%)",
"Multiple listings for same product with varying colors",
"Repetitive and poorly formatted key highlights",
"Seller is Flipkart (trusted marketplace)",
"Product description lacks detailed specifications"
]
}
```

Appendix D - Evidence Ledger Categories

Category	Example Evidence to Surface	Caution
Seller	Seller age, seller identity, fulfilment information when present	Do not infer unavailable seller history
Review	Repeated phrases, burst timing, rating concentration	Patterns are indicators, not proof of coordinated fraud
Price	Displayed discount or unusual price positioning	Requires comparison baseline for stronger conclusions
Listing	Inconsistent title, brand, or product details	Marketplace layout may create extraction noise

Appendix E - Codebase Module Reference

File Path	Responsibility
lib/main.dart	App entry point, environment variable loading via flutter_dotenv, and app initialization.
lib/screens/home_screen.dart	Main UI screen that handles URL input, loading states, result display, and share intent handling.
lib/services/scraper_service.dart	Fetches product page data from ScraperAPI and strips non visible HTML content before sending it to the LLM.
lib/services/llm_service.dart	Sends the cleaned product content to the Groq API and parses the JSON response.
lib/models/authenticity_result.dart	Model class representing the authenticity analysis response.
lib/theme/brutalist_theme.dart	Custom visual styling for the app's harsh, industrial dashboard look.